

Report - Thomas - v2

September 2025

1 formulas

$$\mathcal{A}_{born}(q') \propto \tilde{T}_1(q') \tilde{T}_2(q') \quad (1)$$

$$\chi(s, b) = \frac{1}{s} \int_0^{0.2} q' dq' J_0(bq') \mathcal{A}_{born}(s, t), \quad -t = (q')^2 \quad (2)$$

$$= \frac{1}{s} \int_0^{0.2} q' dq' J_0(bq') \mathcal{A}_{born}(s, -(q')^2) \quad (3)$$

$$\mathcal{A}_{eik}(s, t) = is \int_0^{30} b db J_0(qb) \left[1 - e^{i\chi(s, b)} \right] \quad (4)$$

para secao de choque total eiconal tomemos $q = 0$ e $q' \neq 0$.

2 Metodo via soma de retangulos

2.1 Codigo

```
def full_int(mg, a1, a2, m2_func, q_val, sqrt_s):  
  
    def integrand(y, x, mg, a1, a2, m2_func, q_val):  
        k = sqrt_s * x  
        phi = 2*np.pi*y  
        jacobian = 2*np.pi*sqrt_s  
        return k*(T_1(k, q_val, phi, mg, a1, a2, m2_func) -  
                  T_2(k, q_val, phi, mg, a1, a2, m2_func))*jacobian  
  
    def inner_integral(x):  
        return fixed_quad(  
            lambda y: integrand(y, x, mg, a1, a2, m2_func, q_val),  
            0, 1, n=n_points  
        )[0]  
  
    integral_value = fixed_quad(inner_integral, 0, 1, n=n_points)[0]  
  
    return integral_value  
  
step = 100  
  
lst_amp_eik = []  
  
lst_q_integration = np.linspace(0, 0.2, 100)  
lst_b_integration = np.linspace(0, 30, 100)
```

```

lst_sqrt_s = np.arange(100, 13000, step)

for sqrt_s in lst_sqrt_s:

    s = sqrt_s ** 2

    eik_amp_sum = 0

    for b_value in lst_b_integration:
        chi_sum = 0

        for q_int in lst_q_integration:

            integral_value = full_int(ensemble_parameters['atlas']['pl']['mg'],
                                      ensemble_parameters['atlas']['pl']['a1'],
                                      ensemble_parameters['atlas']['pl']['a2'],
                                      m2_pl, q_int, sqrt_s)

            diff_t = integral_value

            t = -(q_int**2)

            born_amp_value = amp_calculation(diff_t, sqrt_s**2, ensemble_parameters['atlas']['pl']['a1'], ensemble_parameters['atlas']['pl']['a2'], m2_pl, q_int, sqrt_s)
            chi_value = (q_int * j0(b_value * q_int) * born_amp_value) / s

            chi_sum += chi_value

            factor = 1 - np.exp(1j * chi_sum)
            eik_amp_value = (1j * s) * b_value * j0(b_value * 0) * factor
            eik_amp_sum += eik_amp_value
            lst_amp_eik.append(eik_amp_sum)

```

2.2 Resultado

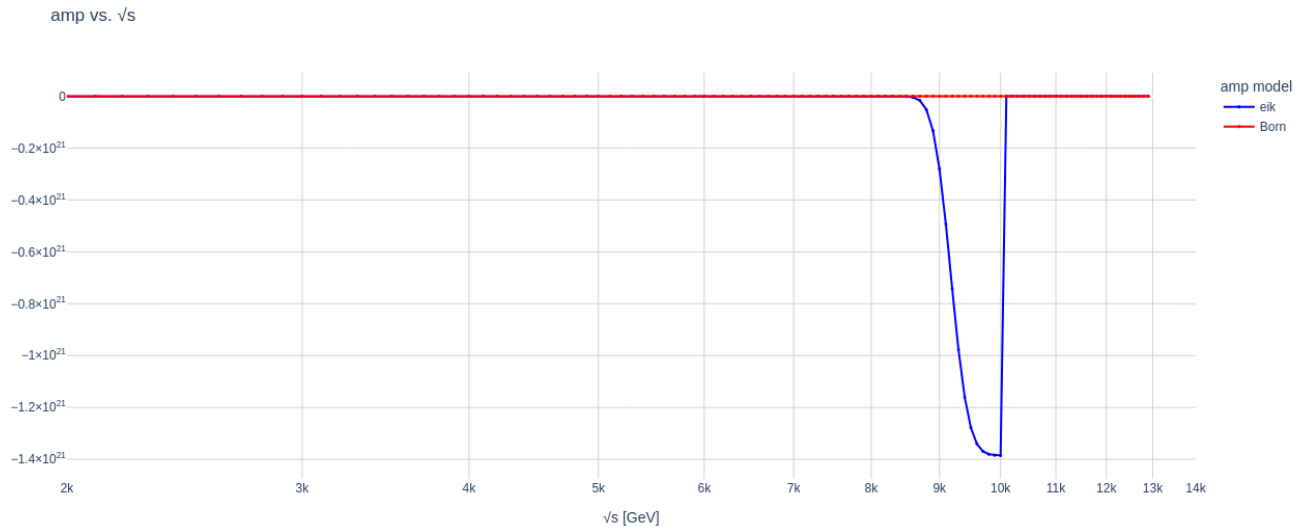


Figure 1: Comparacao de amp born e amp eik via soma de retangulos

3 Metodo via fixed quad

3.1 Codigo

```
def full_int(mg, a1, a2, m2_func, q_val, sqrt_s):

    def integrand(y, x, mg, a1, a2, m2_func, q_val):
        k = sqrt_s * x
        phi = 2*np.pi*y
        jacobian = 2*np.pi*sqrt_s
        return k*(T_1(k,q_val,phi,mg,a1,a2,m2_func) -
                  T_2(k,q_val,phi,mg,a1,a2,m2_func))*jacobian
    def inner_integral(x):
        return fixed_quad(
            lambda y: integrand(y, x, mg, a1, a2, m2_func, q_val),
            0, 1, n=n_points
        )[0]

    integral_value = fixed_quad(inner_integral, 0, 1, n=n_points)[0]

    return integral_value

def chi_integrand(q, b_value, sqrt_s, mg, a1, a2, m2_pl, eps):
    s = sqrt_s ** 2
    t = -(q**2)
    integral_value = full_int(mg, a1, a2, m2_pl, q, sqrt_s)
    born_amp = amp_calculation(integral_value, s, eps, t)
    chi_value = (q * j0(b_value * q) * born_amp) / s
    return chi_value

def eik_integrand(b, sqrt_s, mg, a1, a2, m2_pl, eps, n_points):
    s = sqrt_s ** 2

    # Integrate over q to get chi_sum
    chi_sum, _ = fixed_quad(
        lambda q: chi_integrand(q, b, sqrt_s, mg, a1, a2, m2_pl, eps),
        0.0, 0.2, n=n_points
    )

    factor = 1 - np.exp(1j * chi_sum)
    eik_value = (1j * s) * b * j0(b * 0) * factor

    return eik_value

# Parameters
step = 100
n_points = 5000

# Main loop
lst_amp_eik = []
lst_sqrt_s = np.arange(100, 13000, step)

for sqrt_s in lst_sqrt_s:

    # Integrate over b
    eik_amp, _ = fixed_quad(
        lambda b: eik_integrand(
            b, sqrt_s,
            ensemble_parameters['atlas']['pl']['mg'],
            ensemble_parameters['atlas']['pl']['a1'],
```

```

ensemble_parameters['atlas']['pl']['a2'],
m2_pl,
ensemble_parameters['atlas']['pl']['epsilon'],
n_points
),
0.0, 30.0, n=n_points
)

lst_amp_eik.append(eik_amp)

```

3.2 Resultado

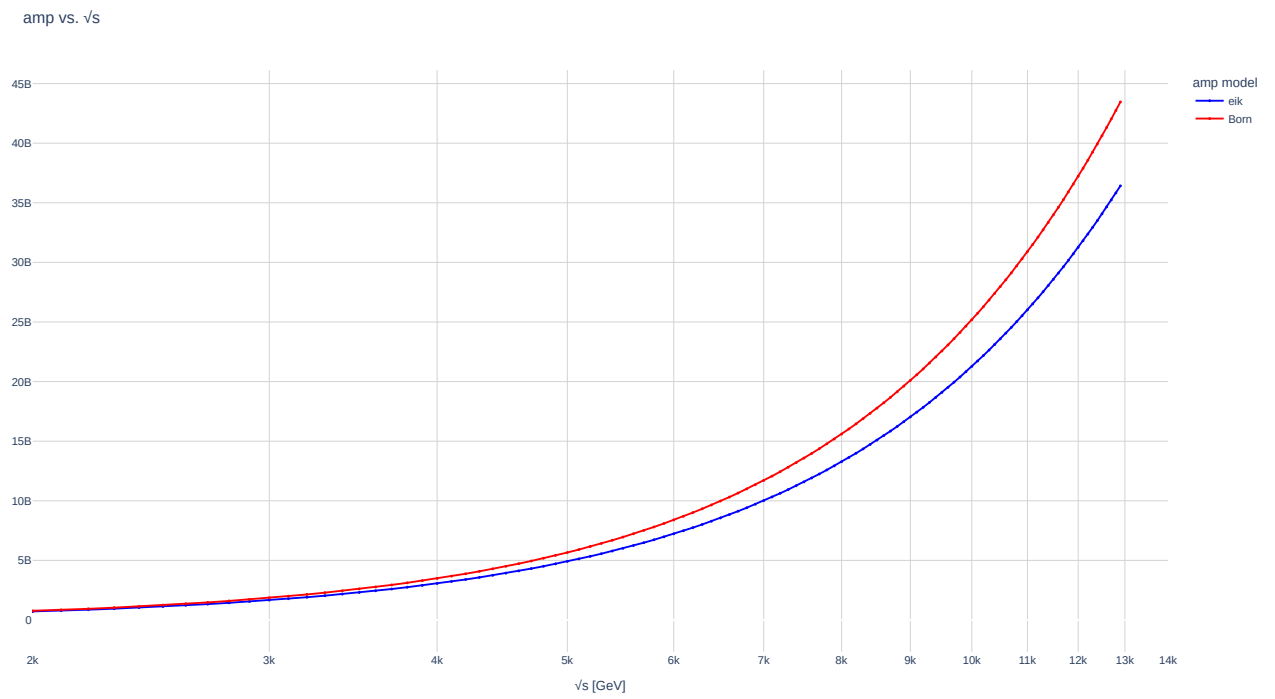


Figure 2: Comparacao de amp born e amp eik via fixed quad